Firjan SENAI

Específico Profissional 2

Desenvolvimento de Sistemas

• LINGUAGENS DE PROGRAMAÇÃO • APIs E FRAMEWORKS
•INTEGRAÇÃO DE SISTEMAS • CONTROLE DE VERSÃO

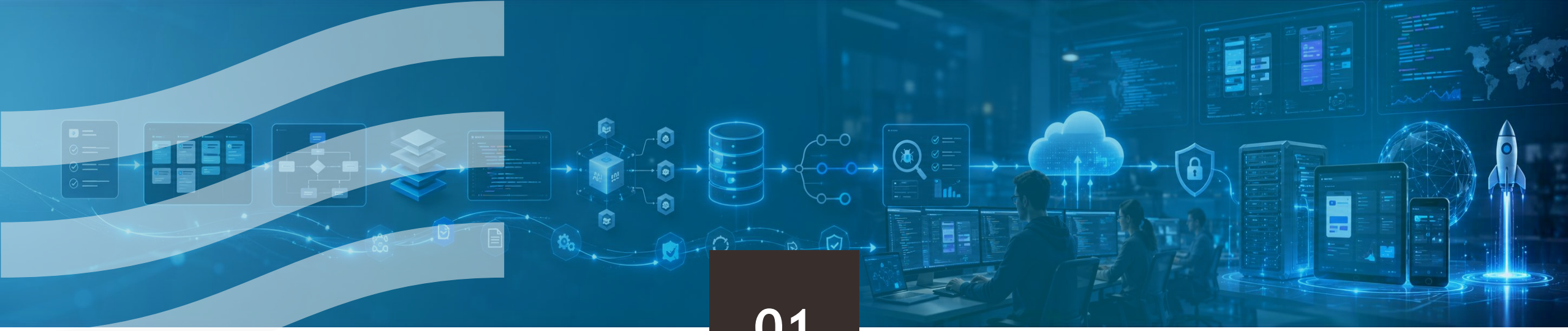


2026



Sumário

1. **Fundamentos do Desenvolvimento de Sistemas**
   Visão sistêmica, planejamento, organização do trabalho e metodologias de desenvolvimento
2. **Qualidade e Requisitos do Sistema**
   Qualidade, integridade, usabilidade, segurança da informação e requisitos não funcionais
3. **Linguagens de Programação**
   Ferramentas, boas práticas e desenvolvimento multiplataforma
4. **APIs, Bibliotecas e Frameworks**
   Construção de rotinas de software e integração de recursos
5. **Integração de Sistemas**
   Comunicação entre subsistemas, plataformas e serviços
6. **Padrões de Projeto**
   Design Patterns, arquitetura e organização do código
7. **Controle de Versão**
   Gerência de configuração, repositórios, versionamento e rastreabilidade
8. **Documentação de Sistemas**
   Documentação técnica, registro das atividades e organização das informações



01

Terminal

Fundamentos de TI para Desenvolvimento de Sistemas
Instrutor: Vitor Hugo Almeida de Sousa

Firjan SENAI

Navegando pelo Terminal no Windows

O Terminal é uma interface de linha de comando que permite executar tarefas de forma direta e rápida.
Com ele, é possível navegar entre pastas, abrir arquivos e executar programas.

O Terminal é uma ferramenta útil para interagir com o sistema operacional de maneira rápida.
Comandos como cd, dir e cls ajudam o usuário a navegar e organizar os arquivos de forma mais eficiente.
Aprender esses comandos é essencial para o trabalho com desenvolvimento, redes e automações.

- cd – Muda de diretório
- dir – Lista conteúdos
- cls – Limpa a tela
- exit – Sai do terminal

Executando Comandos no Terminal

Aprenda na prática

Agora que você já conhece alguns comandos básicos, vamos praticar! Observe os exemplos ao lado e tente replicar os mesmos passos no seu computador. Cada comando possui uma função específica para interagir com arquivos e pastas.

- cd nome_da_pasta – Acessa a pasta desejada
- cd .. – Volta uma pasta
- dir – Mostra o que tem dentro da pasta atual
- cls – Limpa tudo o que aparece na tela
- exit – Fecha o terminal

Faça você mesmo

- Navegue até a pasta C:
- Navegue até a pasta Desktop dentro do seu usuário
- Liste tudo que existe dentro da pasta Desktop
- Limpe o terminal
- Feche o terminal



Manipulando Pastas pelo Terminal no Windows

O Terminal também permite criar, renomear, mover e excluir pastas de forma direta e rápida. Com ele, é possível organizar seus diretórios de maneira eficiente e automatizada.

Comandos como mkdir, ren, move e rmdir ajudam o usuário a gerenciar pastas de forma prática e objetiva. Aprender esses comandos é essencial para o trabalho com desenvolvimento, redes e automações.

- mkdir – Cria uma nova pasta
- ren – Renomeia uma pasta existente
- move – Move uma pasta para outro local
- rmdir – Remove uma pasta vazia
  - rmdir /s – quando tem subpastas
  - rmdir /s /q – apaga sem perguntar
- del – excluir arquivos
- edit – cria um arquivo e já abre para edição
- type – mostra conteúdo de um arquivo de texto

Manipulando Pastas no Terminal

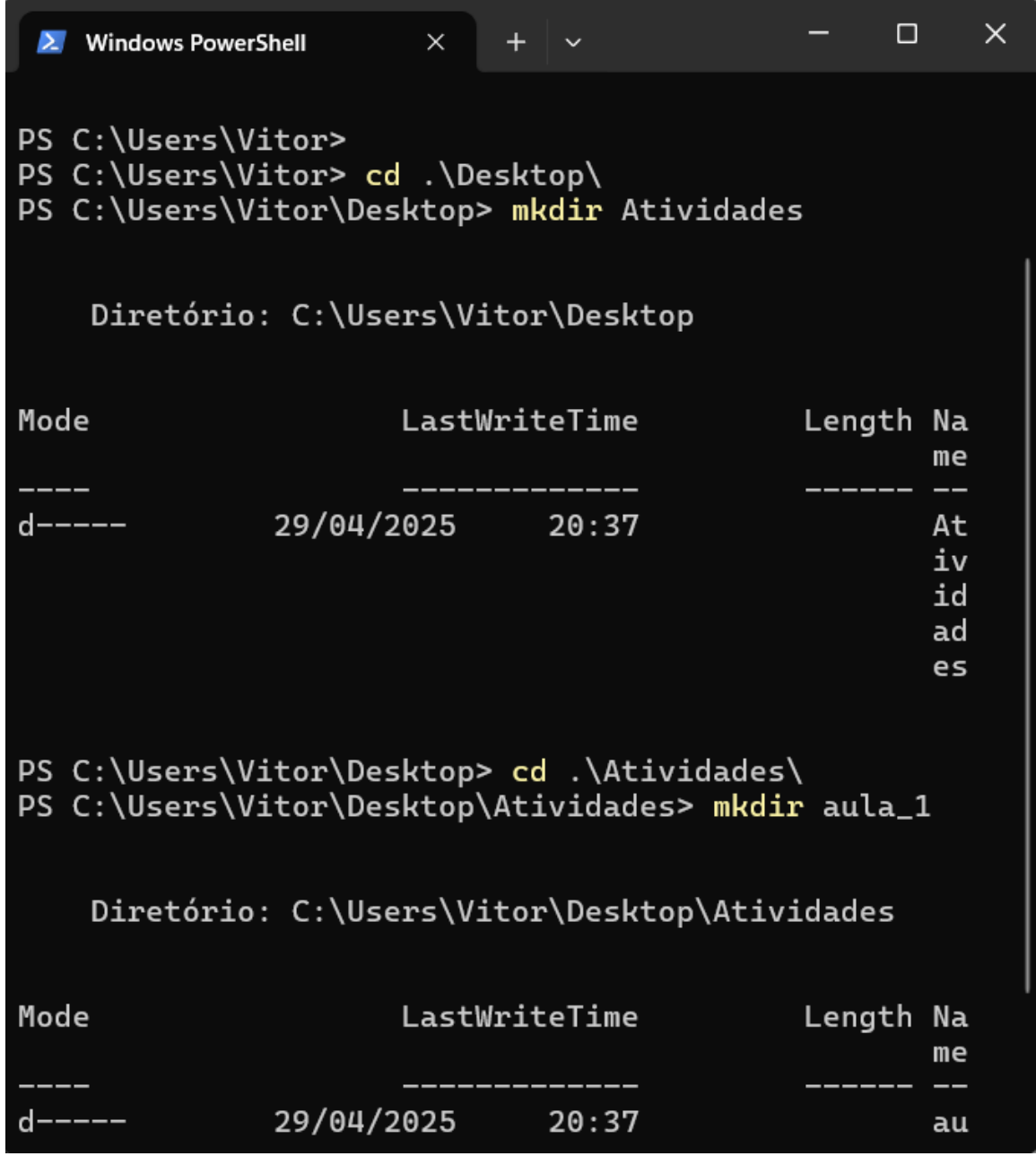
Comandos básicos

Esses comandos são essenciais para explorar, acessar e manipular pastas no seu computador.

- mkdir – Cria uma nova pasta
- ren – Renomeia uma pasta existente
- copy – Copiar um arquivo
- move – Move uma pasta para outro local
- rmdir – Remove uma pasta vazia

Faça você mesmo

- Navegue até o Desktop
- Crie uma pasta em Desktop chamada “Atividades” e outra pasta chamada aula_3
- Entre na pasta “Atividades” e crie dentro dessa pasta duas pasta chamada aula_1 e aula_3
- Renomeie a pasta aula_3 para aula_2
- Volte para pasta Desktop e mova a pasta aula_3 para pasta “Atividades”
- Copia a pasta “Atividades” para a pasta Documentos
- Ao final, remova a pasta “Atividades”



Fontes de imagem – Criada pelo autor

Copiando Pastas e Arquivos pelo Terminal no Windows

O Terminal também permite copiar pastas e arquivos. Esse comando exige um pouco mais de detalhes, uma vez que por padrão não copia subpasta e podem existir intervenções necessárias no processo, como confirmações.

Aprender esses comandos é essencial para o trabalho com desenvolvimento, redes e automações.

- xcopy – Copiar pastas e arquivos
  - xcopy <origem> <destino> /E /I /H /Y
    - /E: copia todas as subpastas, inclusive as vazias.
    - /I: assume que o destino é uma pasta, mesmo que ela ainda não exista.
    - /H: copia arquivos ocultos e de sistema também.
    - /Y: suprime a confirmação ao sobrescrever arquivos existentes.

Terminal

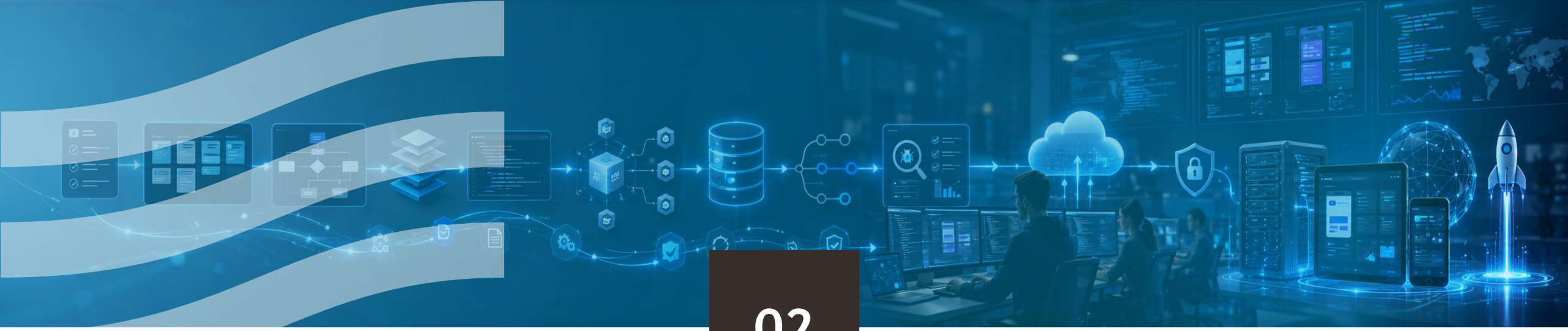
A importância do terminal na rotina do programador

Saber utilizar o terminal é uma habilidade fundamental para qualquer programador. Mesmo em tempos de interfaces gráficas, muitos processos são mais rápidos e eficientes pela linha de comando.

Atenção



Ter domínio básico do terminal permite que o programador automatize tarefas, navegue por diretórios rapidamente e use ferramentas poderosas como Git, SSH e gerenciadores de pacotes. Além disso, é essencial para trabalhar com servidores, ambientes de desenvolvimento e depuração de sistemas. Mesmo comandos simples podem otimizar significativamente o dia a dia de quem programa.



02

GIT

Desenvolvimento de Sistemas

Instrutor: Vitor Hugo Almeida de Sousa

Firjan SENAI

Git: controle de versão para desenvolvedores

O que é o Git?
Ferramenta essencial para controle de versão!
Git é um sistema de controle de versão distribuído que permite que várias pessoas trabalhem em um mesmo projeto de forma segura. Com ele, é possível salvar o histórico de alterações de arquivos, comparar versões e até mesmo voltar para estados anteriores do projeto. É muito usado em desenvolvimento de software para facilitar a colaboração e evitar a perda de código.

Git é uma ferramenta poderosa para acompanhar mudanças no código-fonte ao longo do tempo. Ele permite que você registre cada alteração feita, quem fez, e por que fez. Além disso, com o Git é possível trabalhar em diferentes funcionalidades simultaneamente por meio de ramificações (branches), e depois unir (merge) tudo ao projeto principal com segurança.

- Git registra cada alteração no projeto.
- Permite trabalhar em equipe com segurança.
- Facilita testes e recuperação de versões anteriores.

Instalando e Configurando o Git

Primeiros passos com o Git

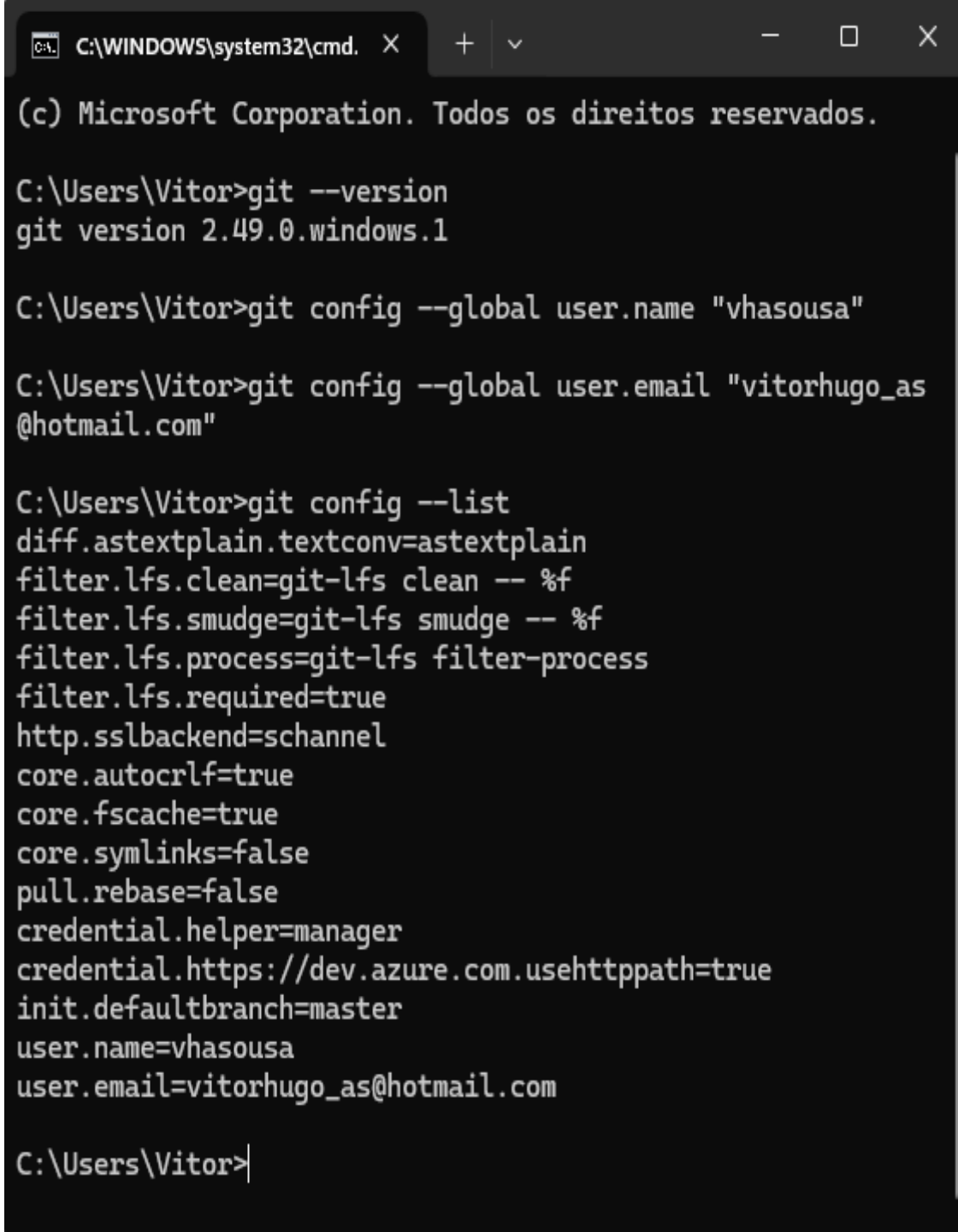
Agora que você está começando a trabalhar com versionamento de código, é essencial instalar e configurar o Git corretamente no seu computador. A instalação é simples e a configuração inicial garante que seus registros de alterações tenham autoria identificada.

- Verifique se a instalação está funcionando
  - Abra o terminal e digite:
    - git --version

- Configure seu nome de usuário
  - Digite no terminal:
    - git config --global user.name "Seu Nome"

- Configure seu e-mail
  - Digite no terminal:
    - git config --global user.email "seuemail@exemplo.com"

- Verifique as configurações
  - Digite:
    - git config --list



Firjan SENAI

Conceitos básicos

Primeiros passos com o Git

Comandos necessários para criar repositório local e rastrear as alterações de arquivos em um projeto.

- Iniciar repositório
  - git init

O Git monitora os arquivos do seu projeto em diferentes **estágios**, cada um com uma função:

- Untracked: Arquivo existe na pasta, mas o Git ainda não o está rastreando
- Staged: Arquivo foi adicionado com git add e está pronto para ser salvo
- Commited: Arquivo foi salvo no histórico do Git com **git commit**

```
C:\WINDOWS\system32\cmd.

C:\Users\VHSOUSA.SISTEMAFIRJAN>cd Desktop

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop>mkdir projeto-git

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop>git init
Initialized empty Git repository in C:/Users/VHSOUSA.SISTEMAFIRJAN/Des
ktop/.git/

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop>edit ola.txt

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop>git add ola.txt

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop>git commit -m "Versao1"
[master (root-commit) 0cb01db] Versao1
 Committer: Vitor Hugo Almeida De Sousa <vhsousa@firjan.com.br>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

 1 file changed, 1 insertion(+)
 create mode 100644 ola.txt

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop>
```

Repositório: Guardando e Organizando Códigos

Um repositório é um local usado para armazenar e organizar arquivos digitais. Na programação, ele serve como um histórico do projeto, permitindo controlar versões, compartilhar alterações e manter o código-fonte seguro.
No Git, o repositório pode ser local, em seu computador, ou remoto, como no GitHub, e registra cada modificação feita, facilitando o trabalho em equipe e a recuperação de versões anteriores.

Saiba Mais!



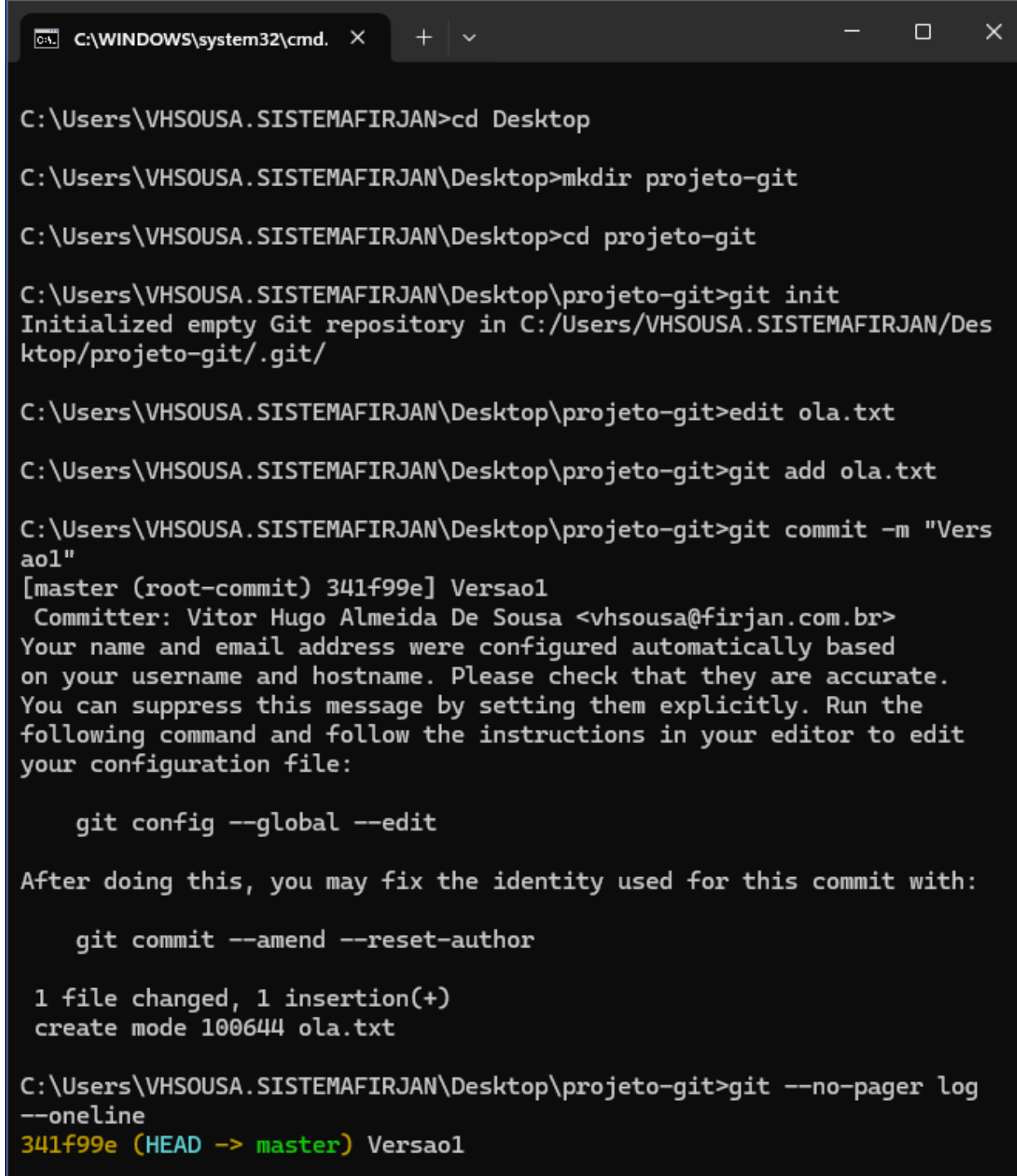
Repositório é onde os arquivos de um projeto são armazenados.
Na programação com Git, ele guarda todas as versões do código, permitindo o controle e o registro das mudanças feitas.
Com isso, é possível trabalhar em equipe de forma segura, acompanhar o histórico e recuperar versões anteriores.

Praticando Git

Primeiro versionamento

Veja o exemplo a seguir e faça você mesmo utilizando o terminal

- Crie em Desktop uma pasta projeto-git
  - mkdir projeto-git
  - cd projeto-git
- Inicie um repositório na pasta projeto-git
  - git init
- Crie um arquivo de texto chamado ola.txt
  - echo > ola.txt
- Abra o arquivo de texto e escreva “Ola, git!”
- Veja o status do arquivo no git
  - git status
- Mude para o status staged e veja o status novamente
  - git add ola.txt
  - git status
- Por fim, salve as alterações no histórico do git
  - git commit -m “Versao 1”
- Veja agora o histórico de commit do Git
  - git --no-pager log --oneline



Praticando Git

Versões diferente de um mesmo arquivo

Agora veremos na prática como funciona o histórico de commit do git veremos o versionamento na prática

- Ainda na pasta projeto-git, abra o arquivo ola.txt e adicione o texto a seguir:
  - “Um novo texto para que eu possa testar o versionamento do git”

- Observe que o status agora voltou para modified, pois houveram mudanças no arquivo
  - git status
- Coloque o arquivo em staging
  - git add ola.txt
- Salve a nova alteração feita
  - git commit –m “Versao 2”
- Observe o histórico atual de commit
  - git log --oneline

```
C:\WINDOWS\system32\cmd.

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>git status
On branch master
nothing to commit, working tree clean

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>edit ola.txt

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory
)
        modified:   ola.txt

no changes added to commit (use "git add" and/or "git commit -a")

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>git add ola.txt

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>git commit -m "Vers
ao2"
[master b9b69cc] Versao2
 Committer: Vitor Hugo Almeida De Sousa <vhsousa@firjan.com.br>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

 1 file changed, 3 insertions(+), 1 deletion(-)

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>git --no-pager log
--oneline
b9b69cc (HEAD -> master) Versao2
341f99e Versao1
```

Firjan SENAI

Praticando Git

Voltando versões do arquivo

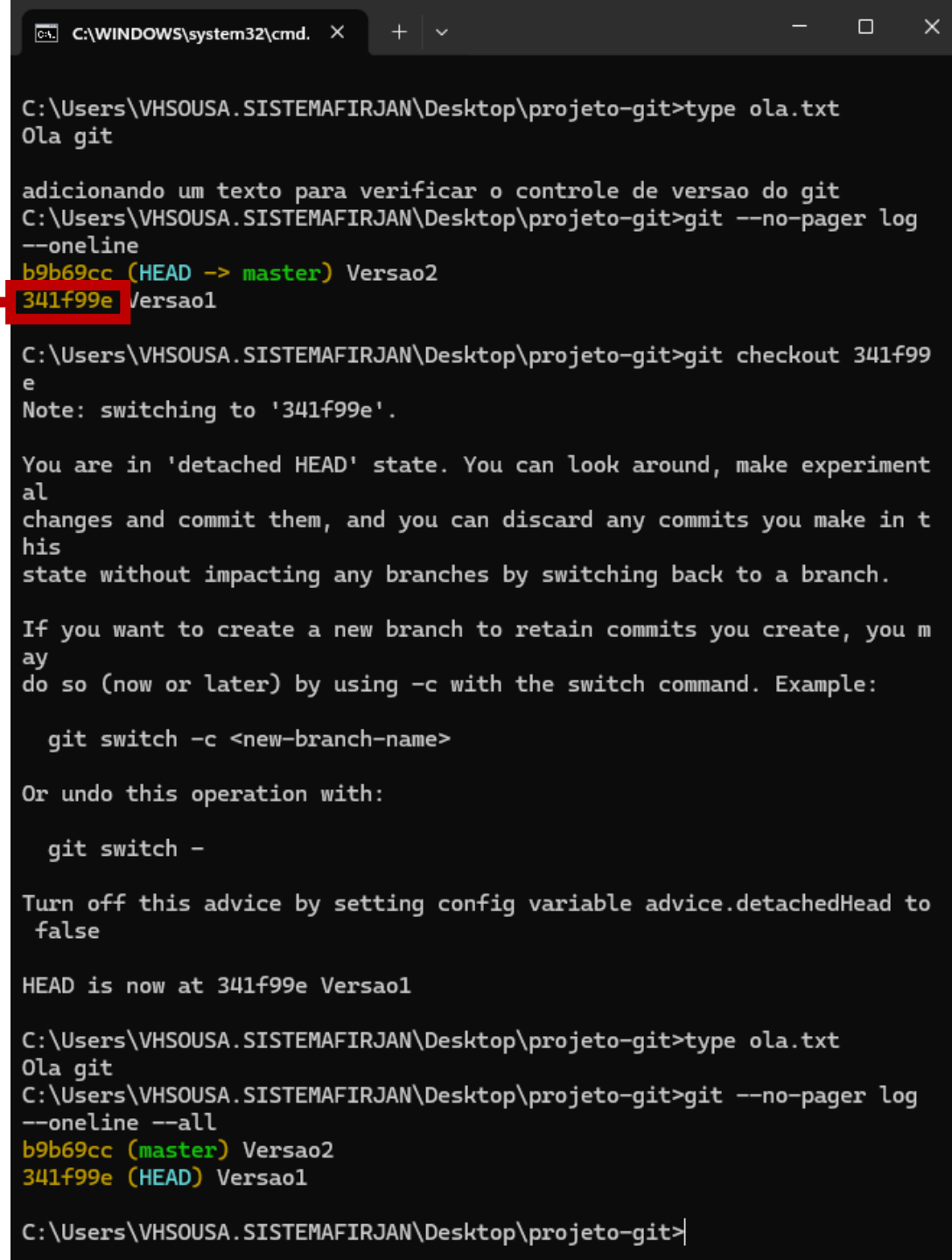
Agora veremos na prática como funciona o histórico de commit do git veremos o versionamento na prática

- Abra o arquivo ola.txt e observe o texto contido nele.
- Feche o arquivo e volte para versão anterior commitada pelo git.
  - git checkout 341f99e

    Atenção que o seu hash estará diferente do meu. Cada commit tem o seu hash único

- Observe o conteúdo do arquivo ola.txt e veja se está como na versão 1, apenas com o texto “Ola, Git!”
- Volte para última Versão do seu código
  - git checkout b9b69cc

Faça mais uma alteração, adicionando mais texto ao seu arquivo ola.txt, coloque o arquivo em stage e depois faça o commit das alterações e observe o histórico de commits



Praticando Git

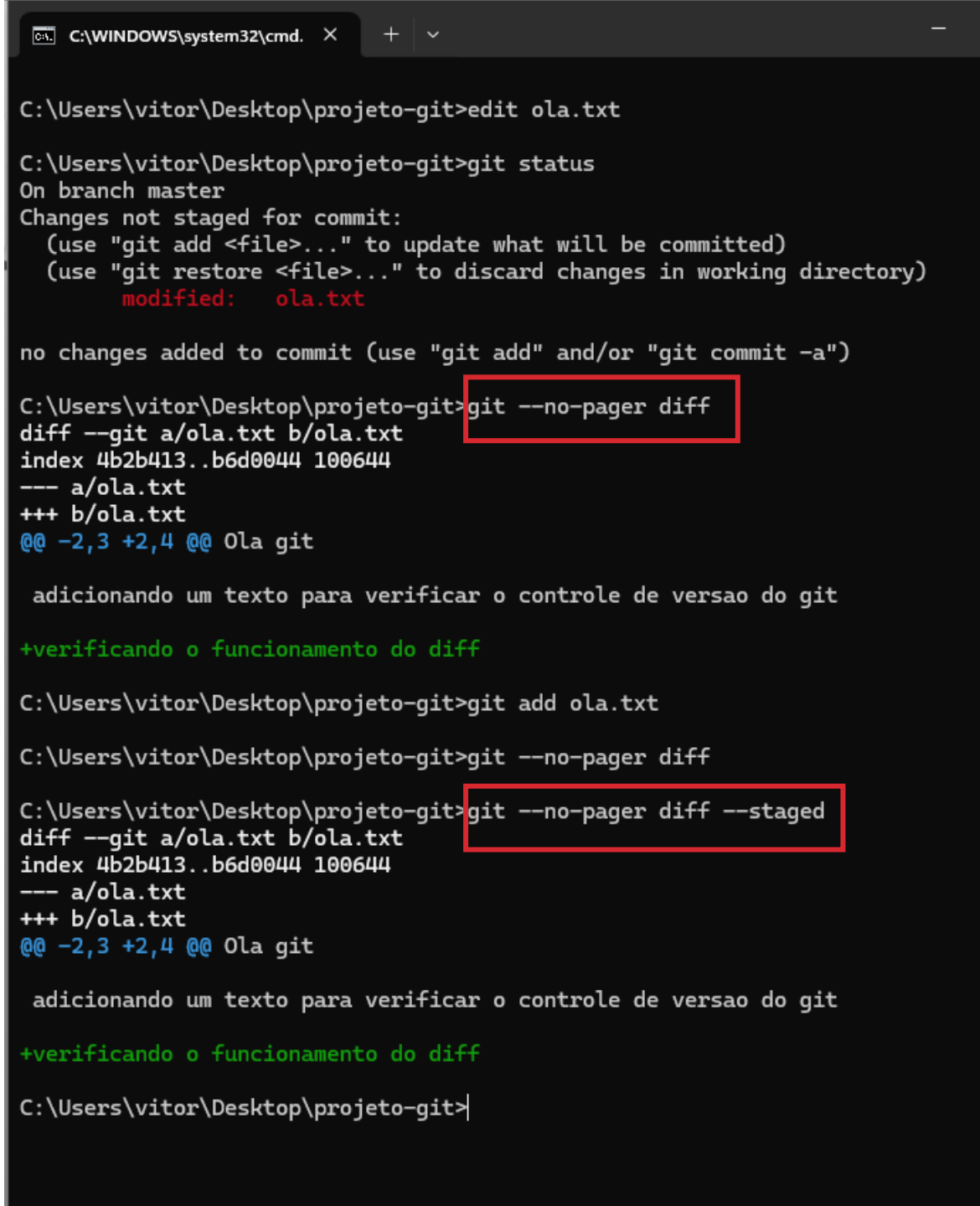
Comparando o arquivo com a última versão

Antes de adicionar ou salvar uma alteração, podemos verificar exatamente o que foi modificado.O comando git diff compara o arquivo atual com a última versão preparada ou registrada pelo Git.

As linhas adicionadas aparecem com o símbolo + e as linhas removidas aparecem com o símbolo -.

- git diff — mostra alterações ainda não adicionadas
- git diff --staged — mostra alterações que já estão em staging

O que observar:
Antes do git add, a alteração aparece com git diff.
Depois do git add, ela aparece com git diff --staged.



Firjan SENAI

Praticando Git

Desfazendo o comando git add

Às vezes adicionamos um arquivo ao staging por engano. Isso não significa que precisamos apagar a alteração.

O comando git reset apenas retira o arquivo da área de preparação. O conteúdo modificado continua no arquivo.git diff — mostra alterações ainda não adicionadas

- O arquivo não é apagado
- A alteração não é perdida
- O arquivo volta para o estado modified
- Um novo git add poderá ser realizado depois

O que observar:
O arquivo sai de “Changes to be committed” e passa para “Changes not staged for commit”.

```
C:\Users\vitor\Desktop\projeto-git>git add ola.txt

C:\Users\vitor\Desktop\projeto-git>git --no-pager diff

C:\Users\vitor\Desktop\projeto-git>git --no-pager diff --staged
diff --git a/ola.txt b/ola.txt
index 4b2b413..b6d0044 100644
--- a/ola.txt
+++ b/ola.txt
@@ -2,3 +2,4 @@ Ola git

 adicionando um texto para verificar o controle de versao do git

+verificando o funcionamento do diff

C:\Users\vitor\Desktop\projeto-git>git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   ola.txt


C:\Users\vitor\Desktop\projeto-git>git reset ola.txt
Unstaged changes after reset:
M       ola.txt

C:\Users\vitor\Desktop\projeto-git>git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ola.txt

no changes added to commit (use "git add" and/or "git commit -a")

C:\Users\vitor\Desktop\projeto-git>type ola.txt
Ola git

adicionando um texto para verificar o controle de versao do git

verificando o funcionamento do diff

C:\Users\vitor\Desktop\projeto-git>
```

Git Reset

Voltando o histórico do projeto

O comando git reset move o repositório para um commit anterior. Seu efeito sobre os arquivos depende do modo utilizado.

O termo HEAD~1 representa um commit antes da posição atual.

- --soft: desfaz o commit, mas mantém as alterações em staging
- --mixed: desfaz o commit e retira as alterações de staging
- --hard: desfaz o commit e também descarta as alterações dos arquivos

Atenção:
O modo --hard pode apagar alterações. Ele deve ser utilizado somente quando houver certeza de que o conteúdo não será mais necessário.

Firjan SENAI

Git Reset

Criando uma nova versão

Para compreender o funcionamento do git reset, criaremos uma nova versão do arquivo.

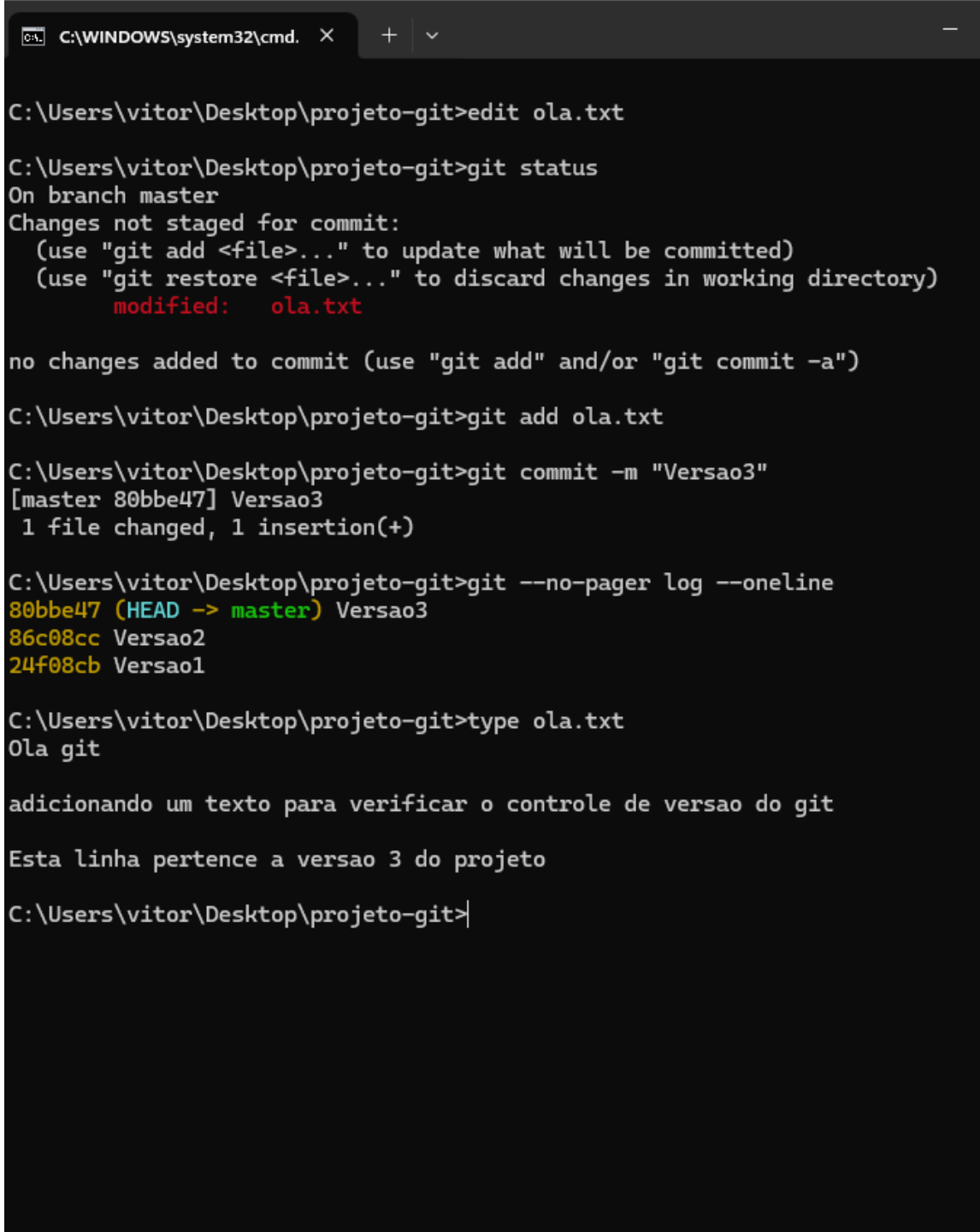
Essa versão será utilizada apenas para testar como um commit pode ser desfeito sem apagar o conteúdo produzido.

Sequência para o CMD:

- edit ola.txt git status
- git add ola.txt
- git commit -m "Versao 3: teste do reset"
- git --no-pager log --oneline
- type ola.txt

Texto para adicionar no arquivo:
Esta linha pertence à versão 3 do projeto.

O que observar:
O commit "Versao 3: teste do reset" deverá aparecer como o primeiro item do histórico.



Git Reset Soft

Desfazendo o commit e mantendo o staging

O modo --soft remove o último commit do histórico, mas conserva todas as alterações na área de staging.

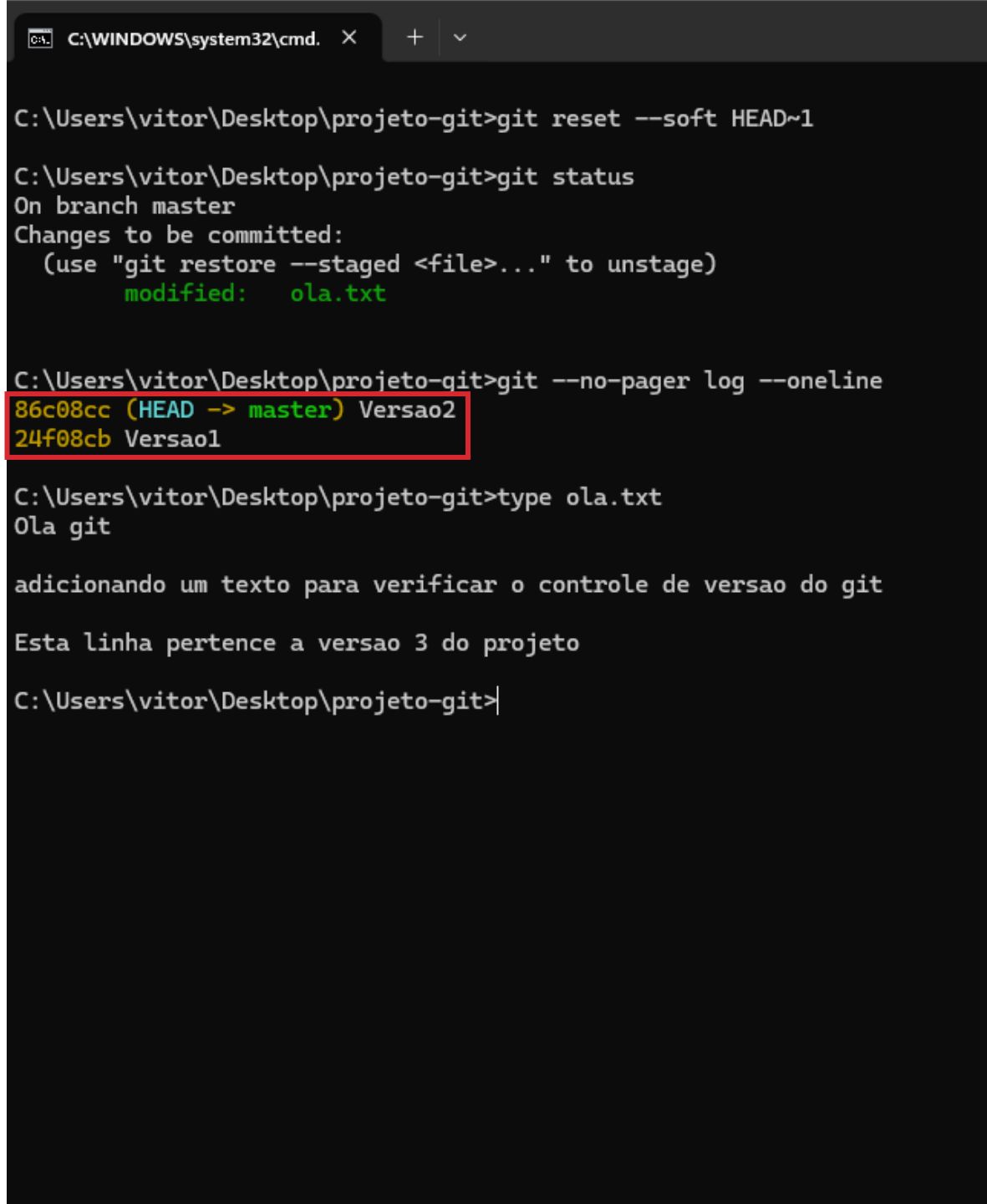
Ele é útil quando percebemos que a mensagem do commit está errada ou quando queremos incluir outra alteração antes de salvar novamente.

Nenhum conteúdo do arquivo será apagado.

Sequência para o CMD:

- git reset --soft HEAD~1
- git status
- git --no-pager log --oneline
- type ola.txt

O que observar:
O commit "Versao 3" não aparece mais no histórico, mas a alteração continua na seção "Changes to be committed".



Firjan SENAI

Git Reset Soft

Corrigindo e registrando a alteração

Depois de utilizar git reset --soft, as alterações permanecem preparadas. Podemos conferir o conteúdo e criar novamente o commit com uma mensagem melhor.

A mensagem deve explicar resumidamente o que foi alterado no projeto.

Sequência para o CMD:

- git status
- git --no-pager diff --staged
- git commit -m "Adiciona linha de teste ao arquivo"
- git --no-pager log --oneline

O que observar:
Um novo commit será criado. Ele terá um hash diferente, mesmo contendo a alteração anterior.

```
C:\Users\vitor\Desktop\projeto-git>git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   ola.txt


C:\Users\vitor\Desktop\projeto-git>git --no-pager diff --staged
diff --git a/ola.txt b/ola.txt
index 4b2b413..964352b 100644
--- a/ola.txt
+++ b/ola.txt
@@ -2,3 +2,4 @@ Ola git

 adicionando um texto para verificar o controle de versao do git

+Esta linha pertence a versao 3 do projeto

C:\Users\vitor\Desktop\projeto-git>git commit -m "Adiciona linha de teste ao arqui
vo"
[master a5aa184] Adiciona linha de teste ao arquivo
 1 file changed, 1 insertion(+)

C:\Users\vitor\Desktop\projeto-git>git --no-pager log --oneline
a5aa184 (HEAD -> master) Adiciona linha de teste ao arquivo
86c08cc Versao2
24f08cb Versao1

C:\Users\vitor\Desktop\projeto-git>
```

Firjan SENAI

Git Reset Mixed

Desfazendo o commit e o staging

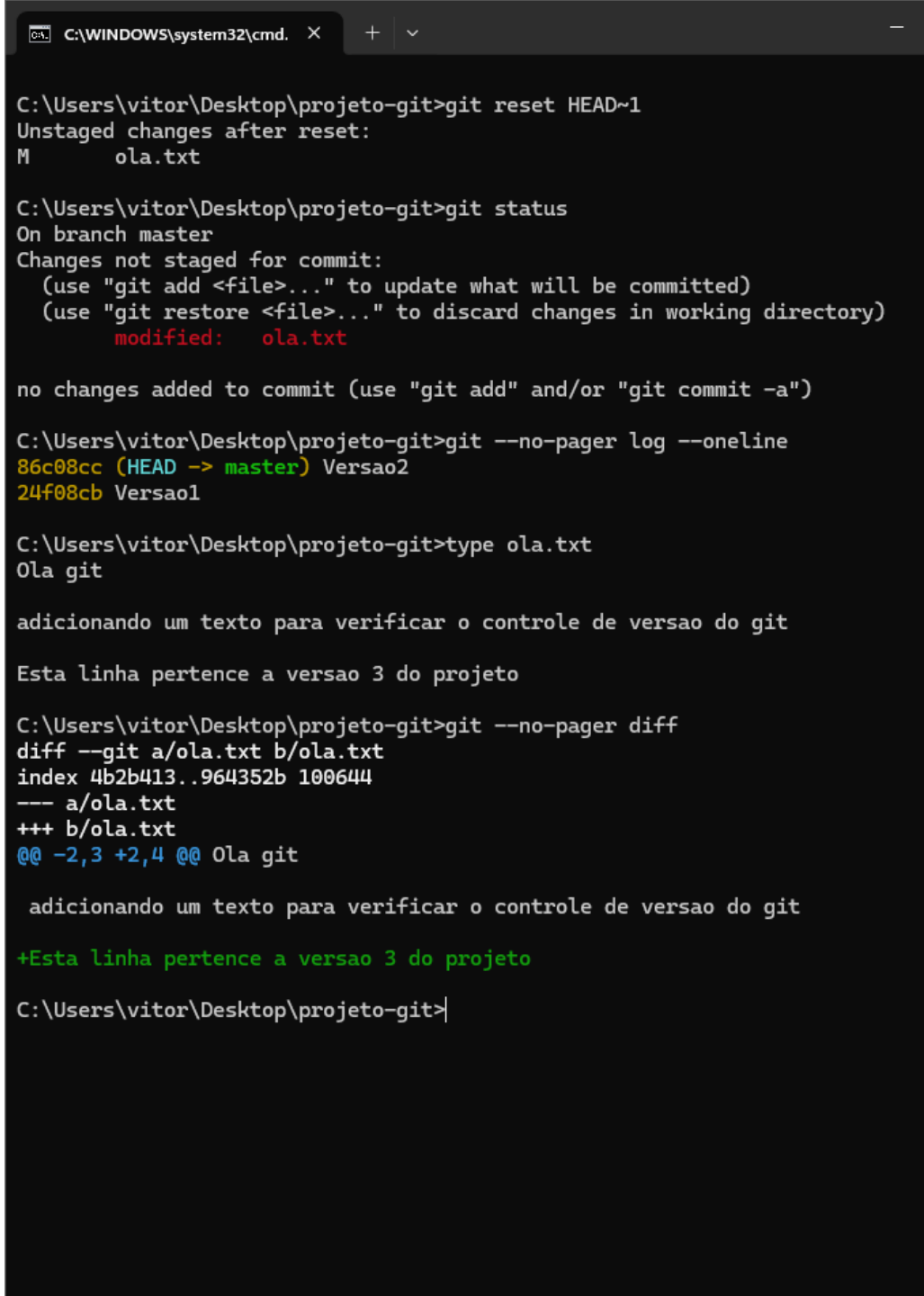
O modo padrão do git reset é o --mixed. Ele desfaz o commit e retira os arquivos da área de staging, mas mantém as alterações no conteúdo.
Após o comando, o arquivo volta para o estado modified.
Os dois comandos abaixo produzem o mesmo resultado:

- git reset HEAD~1
- git reset --mixed HEAD~1

Sequência para o CMD:

- git reset --mixed HEAD~1
- git status
- git --no-pager log --oneline
- type ola.txt
- git --no-pager diff

O que observar:
O commit desaparece do histórico, mas a linha da versão 3 continua dentro do arquivo e pode ser adicionada novamente.



Firjan SENAI

Git Reset Hard

Descartando completamente uma versão

O modo --hard faz o repositório, a área de staging e os arquivos retornarem ao estado de um commit anterior. Isso significa que as alterações posteriores serão descartadas.
Atenção:
Esse comando pode provocar perda de trabalho. Antes de utilizá-lo, verifique o histórico e confirme se as alterações realmente podem ser apagadas.

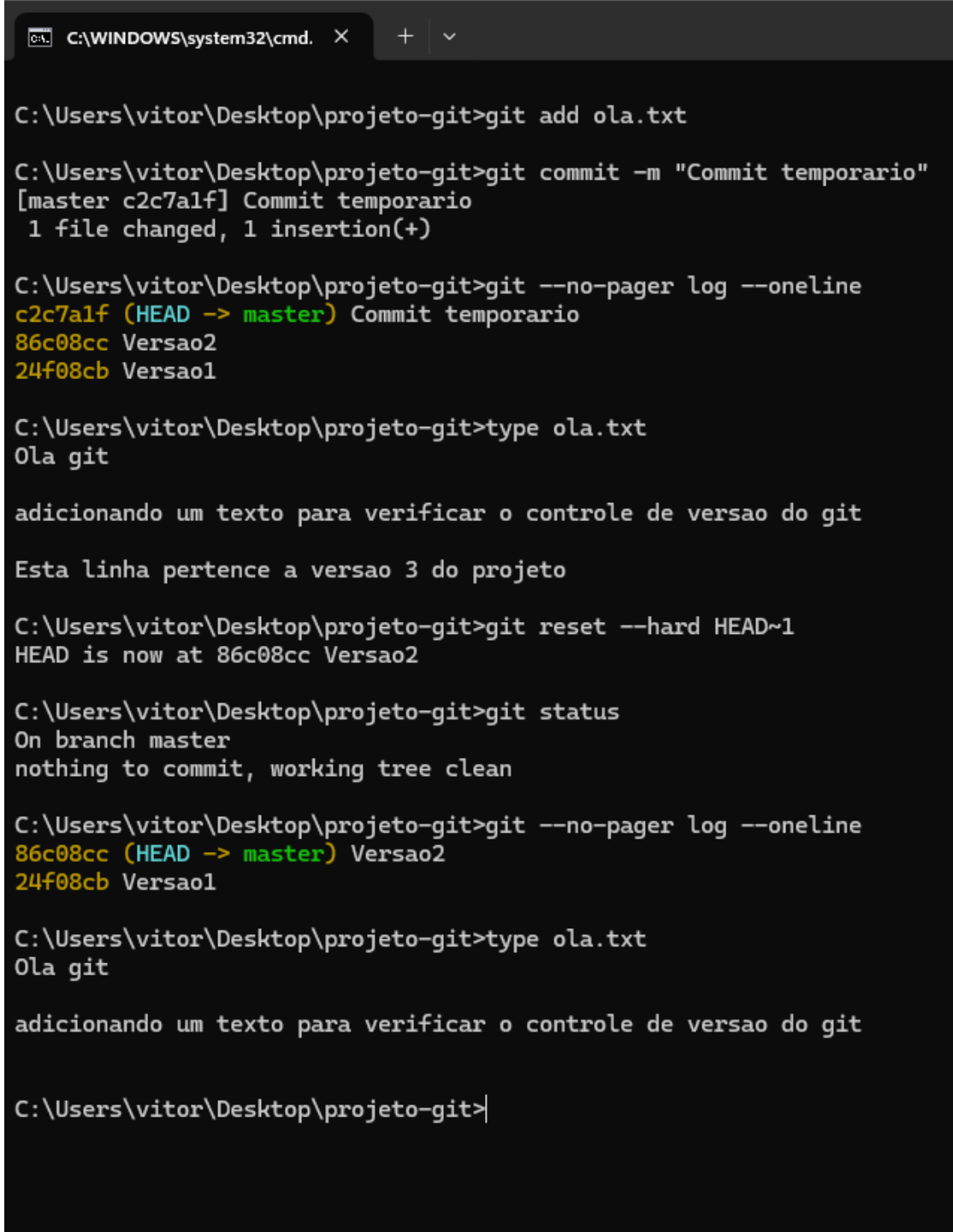
Preparação:

- git add ola.txt
- git commit -m "Commit temporario"
- git --no-pager log --oneline
- type ola.txt

Aplicação do reset:

- git reset --hard HEAD~1
- git status
- git --no-pager log --oneline
- type ola.txt

O que observar:O commit temporário desaparece do histórico e o conteúdo do arquivo retorna à versão anterior.



Git amend

Alterando o commit mais recente

Quando o último commit possui uma mensagem incorreta ou ficou faltando algum arquivo, podemos corrigi-lo utilizando git commit --amend.

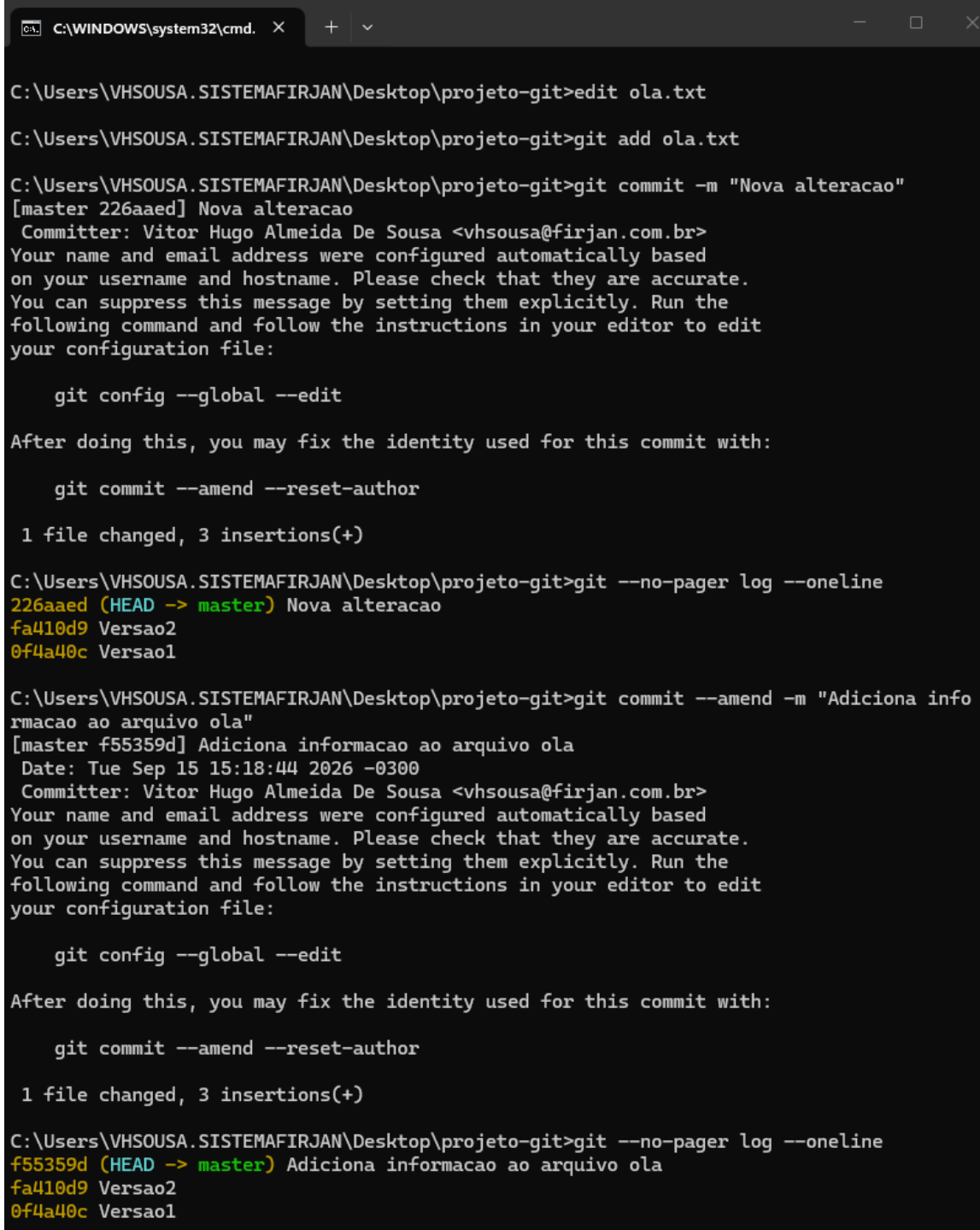
O amend substitui o commit anterior por um novo commit. Por isso, o hash também será alterado.

Utilize esse comando apenas para corrigir o commit mais recente.

Sequência para o CMD:

- edit ola.txt
- git add ola.txt
- git commit -m "Nova alteracao"
- git --no-pager log --oneline
- git commit --amend -m "Adiciona informacao ao arquivo ola"
- git --no-pager log –oneline

O que observar:
A mensagem “Nova alteracao” será substituída e o commit receberá outro hash.



GitIgnore

Utilizando o arquivo .gitignore

Nem todos os arquivos de uma pasta precisam ser versionados. Arquivos temporários, senhas, configurações locais e resultados de compilação normalmente devem ser ignorados.

O arquivo .gitignore informa quais arquivos ou pastas o Git não deve rastrear.

Neste exemplo, todos os arquivos com a extensão .tmp serão ignorados.

Sequência para o CMD:

- echo *.tmp > .gitignore
- git status
- git add .gitignore
- git commit -m "Adiciona arquivo gitignore"
- edit anotacao.tmp
- git status

O que observar: O arquivo anotacao.tmp existe na pasta, mas não aparece entre os arquivos que serão adicionados ao Git.

```
C:\WINDOWS\system32\cmd.

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>edit .gitignore

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>git add .gitignore

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>git commit -m "Adiciona arquivo giti
gnore"
[master 0fad9e7] Adiciona arquivo gitignore
 Committer: Vitor Hugo Almeida De Sousa <vhsousa@firjan.com.br>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

 1 file changed, 1 insertion(+)
 create mode 100644 .gitignore

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>edit anotacao.tmp

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>git status
On branch master
nothing to commit, working tree clean

C:\Users\VHSOUSA.SISTEMAFIRJAN\Desktop\projeto-git>
```

Firjan SENAI

Branches

Criando uma linha separada de desenvolvimento

Uma branch é uma linha separada de desenvolvimento. Ela permite realizar testes ou criar funcionalidades sem alterar imediatamente a versão principal do projeto.

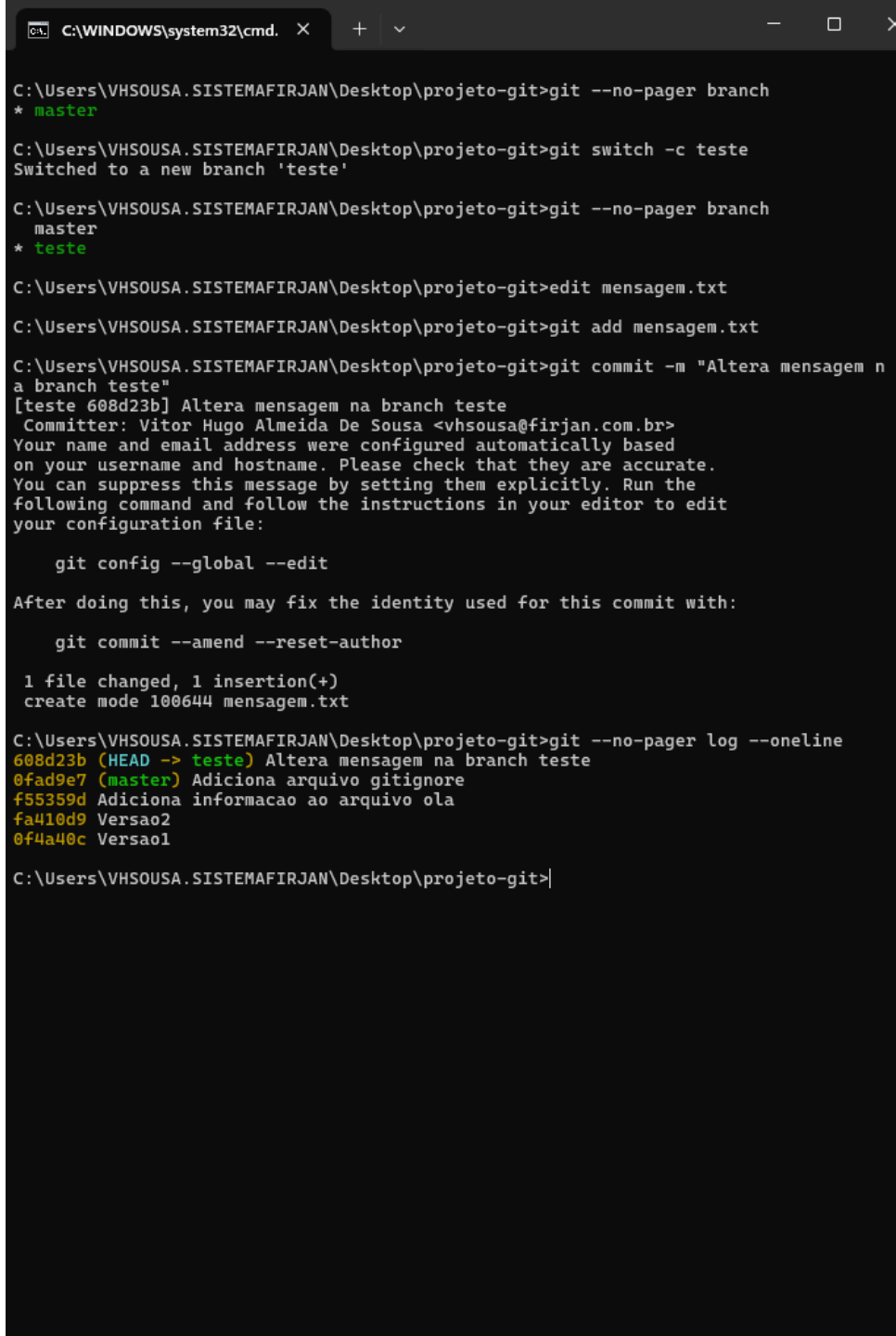
O comando git switch -c cria uma branch e já muda o projeto para ela.

- git branch — lista as branches
- git switch -c nome — cria e acessa uma Branch
- git switch master — retorna para a principal

Sequência para o CMD:

- git --no-pager branch
- git switch -c teste
- git –no-pager branch
- edit mensagem.txt
- git add mensagem.txt
- git commit -m "Altera mensagem na branch teste"
- git --no-pager log –oneline

O que observar: A branch teste aparecerá marcada com um asterisco, indicando que ela está ativa.



Branches

Unindo uma branch

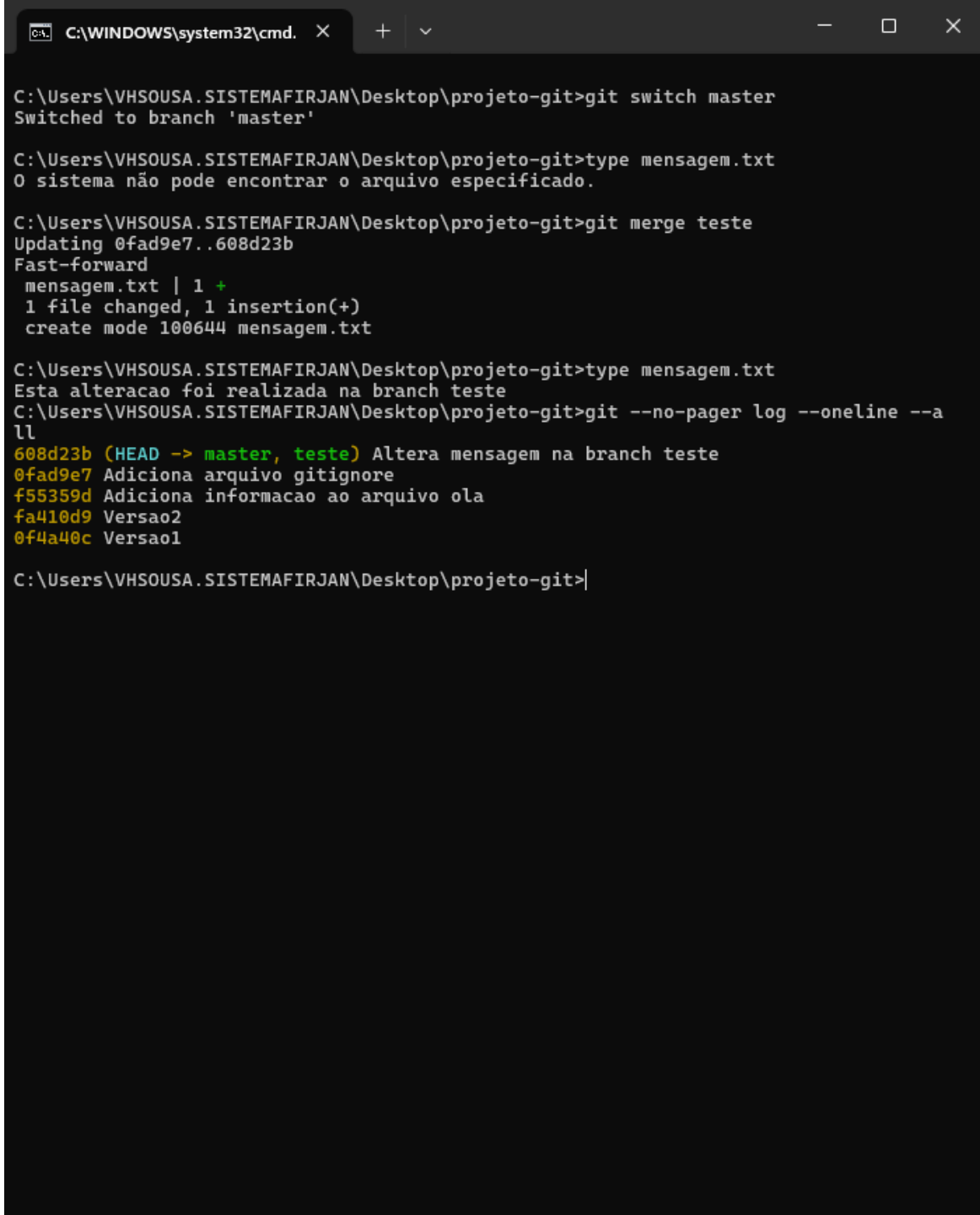
Depois que uma alteração foi testada e aprovada, podemos levá-la para a branch principal utilizando git merge.

Primeiro retornamos para a branch master. Depois solicitamos que o Git una o histórico da branch teste ao histórico principal.

Sequência para o CMD:

- git switch master
- type mensagem.txt
- git merge teste
- type mensagem.txt
- git --no-pager log --oneline --all

O que observar: Antes do merge, a alteração pertence apenas à branch teste. Depois do merge, ela também passa a fazer parte da branch master.



Firjan SENAI

Branches

Removendo uma branch já utilizada

Depois que o conteúdo de uma branch foi unido à principal, a branch de trabalho pode ser removida.

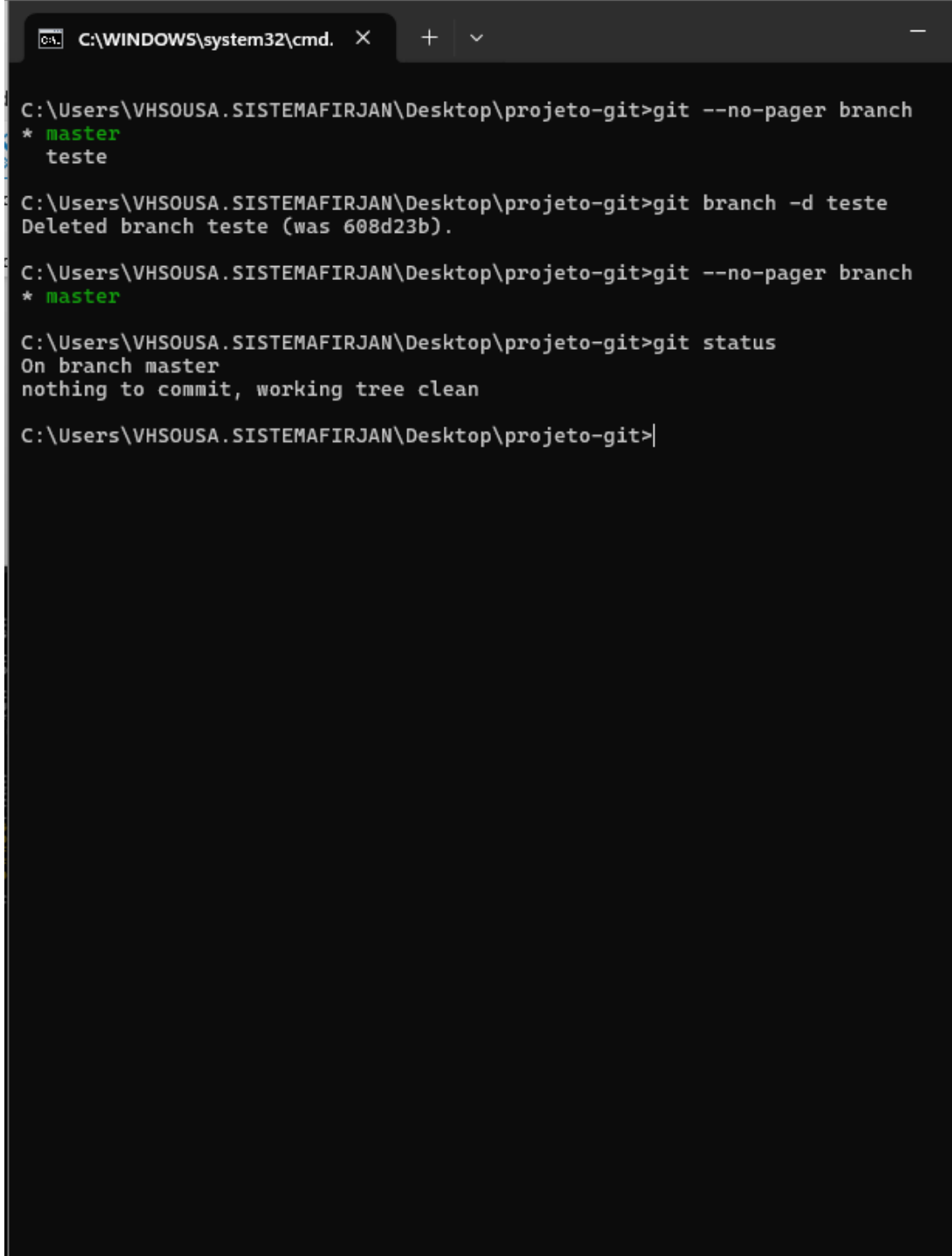
O comando git branch -d realiza uma exclusão segura. O Git impede a remoção quando ainda existem alterações que não foram unidas.

A exclusão da branch não apaga as alterações que já foram incorporadas à branch principal.

Sequência para o CMD:

- Git --no-pager branch
- git branch -d teste
- git branch
- git status

O que observar:
A branch teste deixará de aparecer na lista, enquanto a branch master continuará com o conteúdo incorporado.



Git - Exercício

Praticando git

Crie uma pasta chamada “praticando-git” em Desktop
Inicie o seu repositório Git

Etapas:

- Crie o arquivo autor.txt e escreva seu nome e a turma.
- Adicione o arquivo ao staging. (git add)
- Retire o arquivo do staging sem apagar seu conteúdo. (git reset)
- Adicione novamente e faça um commit.
- Acrescente a data da atividade.
- Utilize git diff para conferir a mudança.
- Faça um segundo commit com a mensagem “versao2”.
- Desfaça o segundo commit utilizando git reset --soft HEAD~1.
- Crie novamente o commit com a mensagem “versao2: adicionando data da atividade”.
- Crie uma branch com o seu primeiro nome.
- Faça uma alteração no arquivo dentro da nova branch e faça o commit.
- Retorne para a branch principal e realize o merge.
- Exiba o histórico completo.
- Remova a branch com seu nome

Firjan SENAI